

Reporte de Práctica

Práctica: Motor DC con PID conectado a InfluxDB y Grafana

Materia de Verano: Internet of Things (IoT) · Ingeniería Mecatrónica · 6.º semestre · Universidad Panamericana

Alumno 1: Eduardo Hernández Torres

Alumno 2: Carlo Gordillo Arenas

Alumno 3: Alejandro Acosta Salazar

Equipo #: 2

Fecha: July 22, 2026

GitHub: <https://github.com/Eduardoht1/IoT>

Tabla de calificación

(uso exclusivo del profesor)

Sección	Obj.	Marco	Mat.	Cód.	Result.	Anál.	Concl.	Total
Puntos	/2	/4	/2	/4	/6	/10	/4	/32

Contents

1	Objetivo	3
2	Marco teórico	3
2.1	Controlador PID en un sistema IoT	3
2.2	El papel de MQTT en el pipeline	3
2.3	InfluxDB como base de datos de series de tiempo	3
2.4	Grafana como herramienta de visualización	4
2.5	Anti-windup en el controlador PID	4
3	Materiales y equipo	5
4	Diagrama de conexiones y arquitectura	5
4.1	Conexiones físicas del hardware	5
4.2	Diagrama de la arquitectura del pipeline	7
5	Código fuente	8
5.1	platformio.ini	8
5.2	main.cpp	8
5.3	Parámetros del PID	12
6	Resultados	13
6.1	Capturas de pantalla del sistema funcionando	13
6.1.1	Serial Monitor del ESP32	13
6.1.2	Data Explorer de InfluxDB con los datos del motor	13
6.1.3	Dashboard completo de Grafana	14
6.2	Consultas Flux utilizadas	14
6.2.1	Panel RPM + Setpoint	14
6.2.2	Panel Error del PID	14
6.2.3	Panel Señal PWM	14
6.3	Experimento de escalón — datos de Grafana	15
6.4	Registro del Serial Monitor	15
7	Análisis y discusión	16
7.1	Análisis del error residual en estado estacionario	16
7.2	Análisis del sobreimpulso	16
7.3	Análisis de la señal PWM	16
7.4	Latencia del pipeline	16
7.5	¿El PID está bien sintonizado? Tu criterio	17
8	Preguntas de análisis técnico	18
9	Conclusiones	19
10	Repositorio y entregables	19

1. Objetivo

Instrucción: Describe en 2–3 oraciones el objetivo de esta práctica con tus propias palabras. Incluye qué se integró, qué se logra monitorear y cuál es la utilidad práctica en un sistema industrial.

El objetivo de esta práctica es la implementación de un control PID en una ESP de manera local, así mismo haciendo uso de un sistema IoT con ayuda de MQTT, Telegraf, Influx y Grafana, lo que nos permitirá visualizar y analizar gráficamente el estado del sistema y el correcto funcionamiento de control.

2. Marco teórico

Instrucción: Explica los conceptos clave con tus propias palabras — no copies definiciones. Cada subsección tiene una pregunta guía para orientarte.

2.1. Controlador PID en un sistema IoT

Pregunta guía: ¿Por qué el PID debe correr en el ESP32 (edge) y no en el servidor, aunque el servidor sea más potente?

Porque el control requiere de una entrada de datos constante y confiable, al llevarlo a la nube, el control no sería capaz de reaccionar rápidamente y estaría muy expuesto a la falla por variaciones en la red.

2.2. El papel de MQTT en el pipeline

Pregunta guía: ¿Qué ventaja tiene publicar por MQTT en lugar de escribir directo a InfluxDB desde el ESP32?

Si estuviera directamente conectada las ESP a influx, en caso de una desconexión, no se podrían seguir enviando datos, en cambio con el uso de MQTT los datos se podrán seguir enviando, aparte de esta forma es más fácil acceder a otros servicios.

2.3. InfluxDB como base de datos de series de tiempo

Pregunta guía: ¿Qué son los Fields y los Tags en InfluxDB? ¿Cuál de los datos del motor es Field y cuál sería Tag?

Fields: Son valores numéricos que mides o mandas y que cambian en el tiempo. Tags: Son los metadatos que se usan para identificar y clasificar datos.

Dato	Field / Tag	Justificación
rpm	Field	Almacena un valor numérico flotante variable que se usa para mediciones y cálculos continuos.
setpoint	Field	Contiene el valor numérico objetivo del sistema que cambia dinámicamente.
error	Field	Representa una métrica calculada numéricamente (resultado matemático de setpoint menos rpm).
pwm	Field	Guarda un valor entero de salida (0 a 255) enviado hacia el actuador.
nodo (nombre del ESP32)	Tag	Identificador textual fijo del dispositivo, utilizado para indexar, filtrar y agrupar la telemetría por fuente.

Table 1: Clasificación de los datos del motor en InfluxDB

2.4. Grafana como herramienta de visualización

Pregunta guía: ¿Grafana almacena datos? ¿Qué ocurre con los datos si Grafana falla por 5 minutos?

No, grafana no los almacena solamente consulta los datos y los muestra por lo que si grafana falla el sistema seguirá funcionando y los datos seguirán en Influx.

2.5. Anti-windup en el controlador PID

Pregunta guía: ¿Qué es el windup del integrador y cómo se ve su efecto en la gráfica de RPM en Grafana?

Es cuando hay una desviación grande entre el valor que se espera en el setpoint y un valor medido; por el motor frenado, con sobrecarga o que se le pide una velocidad máxima que supera la capacidad del sistema, lo que se vería en la grafica como un pico excedente.

3. Materiales y equipo

Instrucción: Enlista todo el hardware y software utilizado. Agrega filas si necesitas.

#	Componente / Software	Cantidad	Observaciones
1	ESP32-DevKitC	1	
2	Motor DC con encoder	1	
3	Driver TB6612FNG	1	
4	Sensor de efecto Hall	1	
5	Fuente de alimentación del motor	1	Voltaje: 12 V
6	Cable USB-A a micro-USB	1	
7	Laptop con VS Code + PlatformIO	1	SO: Windows
8	Docker Desktop	1	Versión: 4.83
9	Mosquitto (broker MQTT)	1	En Docker
10	Telegraf	1	En Docker
11	InfluxDB 2.7	1	En Docker
12	Grafana 10	1	En Docker

4. Diagrama de conexiones y arquitectura

4.1. Conexiones físicas del hardware

Instrucción: Inserta una foto o diagrama de las conexiones del ESP32 con el TB6612FNG, el sensor Hall y el motor. Si lo dibujas a mano, toma una foto y adjúntala descomentando las líneas de abajo.

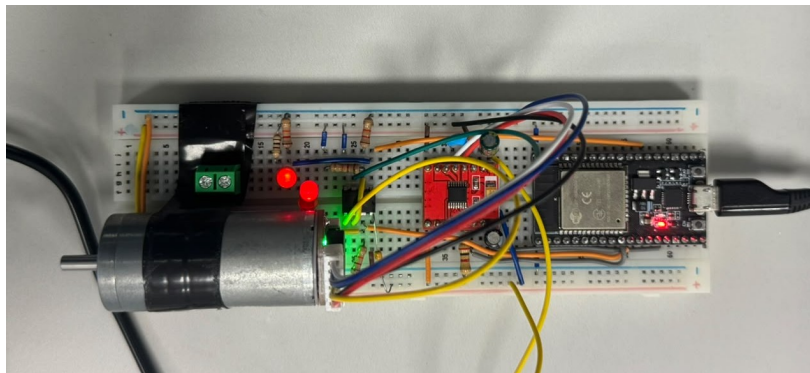


Figure 1: Foto circuito

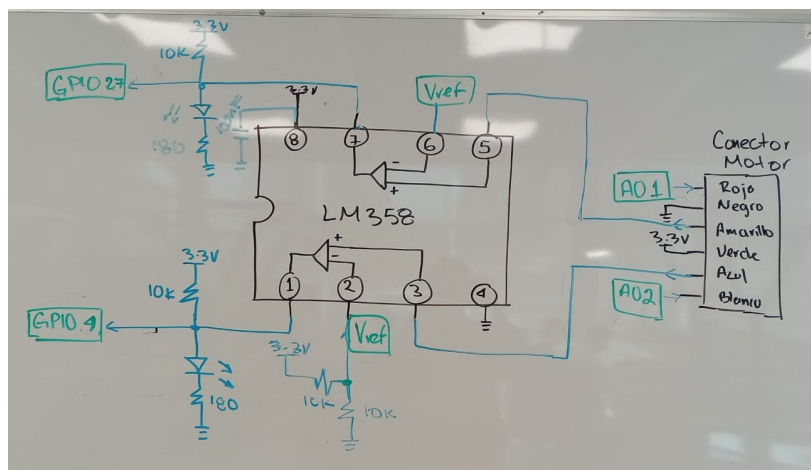


Figure 2: Conexiones OPAMPs y motor

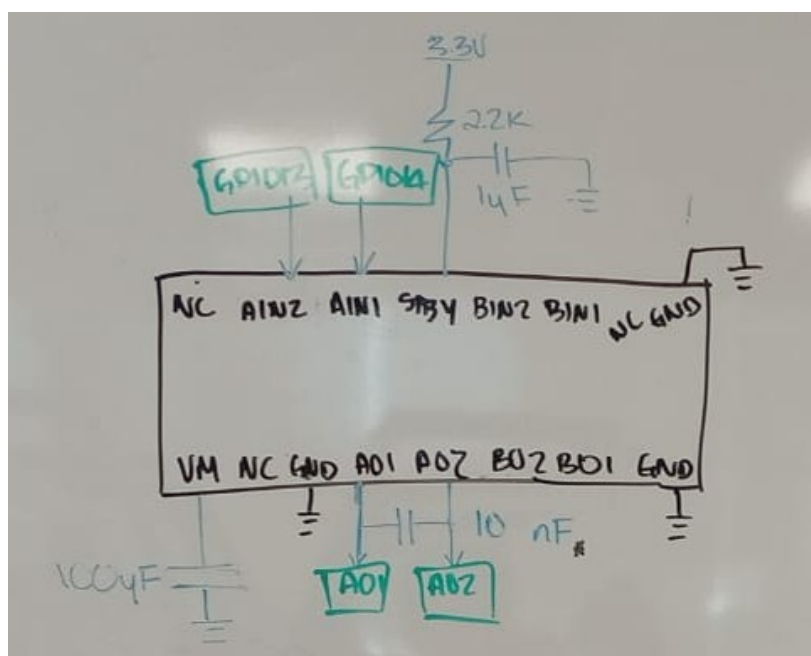


Figure 3: Conexiones puente H

Pin TB6612 / Hall	GPIO ESP32	Función
PWMA	GPIO 26 y GPIO 13	Señal PWM de velocidad
AIN1	GPIO 26	Dirección bit 1
AIN2	GPIO 13	Dirección bit 2
STBY	3.3V (fijo)	Habilitar driver
Hall signal	GPIO 27 y GPIO 4	Pulsos del encoder

Table 2: Tabla de conexiones — completa los GPIO que usaste

4.2. Diagrama de la arquitectura del pipeline

Instrucción: Dibuja el diagrama de bloques del pipeline completo de tu sistema (ESP32 → MQTT → Telegraf → InfluxDB → Grafana) indicando qué dato fluye en cada flecha. Puedes usar el espacio de abajo o insertar una imagen.

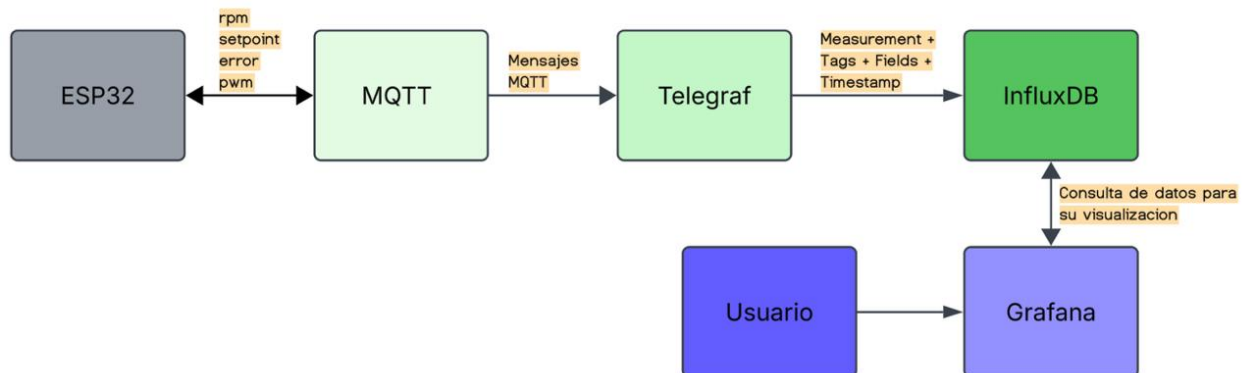


Figure 4: Diagrama de la arquitectura del pipeline

5. Código fuente

Instrucción: Pega el código completo de tu `main.cpp` final. Debe ser el código real que subiste al ESP32, con tus valores de Kp, Ki, Kd y tus GPIOs. No el código de ejemplo.

5.1. platformio.ini

Listing 1: platformio.ini del proyecto

```

1 [env:nodemcu-32s]
2 platform = espressif32
3 board = nodemcu-32s
4 framework = arduino
5 lib_deps =
6     knolleary/PubSubClient      ; Cliente MQTT
7     bblanchon/ArduinoJson      ; Serializar el payload JSON
8 monitor_speed = 115200

```

5.2. main.cpp

Listing 2: main.cpp — PID + MQTT + publicación a InfluxDB

```

1 // Motor DC con PID + publicacion MQTT a InfluxDB/Grafana
2 // Universidad Panamericana - IoT Semana 2
3
4 //docker exec iot-mosquitto mosquitto_pub -h localhost -p 1883 -t "iot/
5   motor/setpoint" -m "10"
6
7 #include <Arduino.h>
8 #include <WiFi.h>
9 #include <PubSubClient.h>
10 #include <ArduinoJson.h>
11
12 // WiFi
13 const char* WIFI_SSID      = "LaptopEdu";
14 const char* WIFI_PASSWORD  = "G65n72R98a02E05";
15
16 // MQTT (IP de la laptop con Docker)
17 const char* MQTT_BROKER    = "192.168.137.1";
18 const int MQTT_PORT        = 1883;
19
20 const char* TOPIC_TELEMETRIA = "iot/motor/telemetry";
21 const char* TOPIC_SETPOINT   = "iot/motor/setpoint";
22
23 // Gpios (Alineados con tu lógica de control)
24 #define AIN2 13 // clockwise (usado como PWM)
25 #define AIN1 26 // counterclockwise
26 #define Amarillo 27 // Encoder Canal A
27 #define Azul 4 // Encoder Canal B
28
29 // Relación de transmisión y encoder
30 const float PPR = 11.0;
31 const float GEAR_RATIO = 35.0;
32 const float REAL_PPR = PPR * GEAR_RATIO;

```



```

33 // Constantes del Controlador PI
34 float Kp = 1.5;
35 float Ki = 12.5;
36
37 // Variables del PID
38 float setpoint = 70.0; // Velocidad objetivo inicial
39 float rpm = 0.0;
40 float integral = 0.0;
41 float pwmSalida = 0.0;
42
43 // Variables del sensor Hall
44 volatile long encoderPulses = 0;
45 portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;
46
47 // Temporizadores
48 const int interval = 20; // Tiempo de muestreo del PID (20 ms)
49 const float dt = interval / 1000.0; // dt en segundos (0.02s)
50 unsigned long previousMillis = 0;
51
52 #define T_MQTT_MS 500 // Publicar a MQTT cada 500 ms
53 uint32_t tMQTT = 0;
54
55 // MQTT
56 WiFiClient wifiClient;
57 PubSubClient mqttClient(wifiClient);
58 bool mqttListo = false;
59
60 // ISR: cuenta pulsos del sensor usando tu lógica direccional
61 void IRAM_ATTR countPulses() {
62     portENTER_CRITICAL_ISR(&mux);
63     if (digitalRead(Azul) > 0) {
64         encoderPulses++;
65     } else {
66         encoderPulses--;
67     }
68     portEXIT_CRITICAL_ISR(&mux);
69 }
70
71 // Lógica de control unificada (Lectura RPM, Error, Anti-Windup y PWM)
72 void ejecutarCicloPID() {
73     // 1. Leer las RPM actuales
74     long currentPulses;
75     portENTER_CRITICAL(&mux);
76     currentPulses = encoderPulses;
77     encoderPulses = 0;
78     portEXIT_CRITICAL(&mux);
79
80     // Calculamos las RPM absolutas
81     rpm = abs(((float)currentPulses / REAL_PPR) * (60.0 / dt));
82
83     // 2. Calcular el error absoluto
84     float error = setpoint - rpm;
85
86     // 3. Calcular la parte Integral con Anti-Windup
87     integral += error * dt;

```

```

88
89 float max_integral = 255.0 / Ki;
90 if (integral > max_integral) integral = max_integral;
91 if (integral < -max_integral) integral = -max_integral;
92
93 // 4. Ecuación del Controlador PI
94 float control_signal = (Kp * error) + (Ki * integral);
95
96 // 5. Saturación (Limitar la señal matemática al mundo real 0-255)
97 int pwm_output = (int)control_signal;
98
99 if (pwm_output > 255) pwm_output = 255;
100 if (pwm_output < 0) pwm_output = 0;
101
102 // 6. Aplicar la potencia al driver del motor
103 digitalWrite(AIN1, LOW);
104 analogWrite(AIN2, pwm_output);
105
106 // Guardar la variable para que se publique en Grafana
107 pwmSalida = (float)pwm_output;
108 }
109
110 // Conectar WiFi
111 void conectarWiFi() {
112     Serial.printf("Conectando a WiFi '%s'...", WIFI_SSID);
113     WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
114     int n = 0;
115     while (WiFi.status() != WL_CONNECTED && n++ < 40) {
116         delay(500);
117         Serial.print(".");
118     }
119     if (WiFi.status() == WL_CONNECTED) {
120         Serial.printf("\n OK IP: %s\n", WiFi.localIP().toString().c_str());
121     } else {
122         Serial.println("\n FALLO WiFi - continuando sin MQTT");
123     }
124 }
125
126 // Callback MQTT: recibir nuevo setpoint desde Grafana
127 void mqttCallback(char* topic, byte* payload, unsigned int len) {
128     String msg = "";
129     for (unsigned int i = 0; i < len; i++) msg += (char)payload[i];
130
131     if (String(topic) == TOPIC_SETPOINT) {
132         float sp = msg.toFloat();
133         if (sp >= 0 && sp <= 700) {
134             setpoint = sp;
135             // Reset integral al cambiar setpoint
136             integral = 0;
137             Serial.printf("<-- Nuevo setpoint: %.1f RPM\n", setpoint);
138         }
139     }
140 }
141
142 // Conectar al broker MQTT

```

```

143 void conectarMQTT() {
144     if (WiFi.status() != WL_CONNECTED) return;
145     int n = 0;
146     while (!mqttClient.connected() && n++ < 5) {
147         Serial.print("Conectando MQTT...");
148         if (mqttClient.connect("ESP32-Motor-PID")) {
149             mqttClient.subscribe(TOPIC_SETPPOINT);
150             mqttListo = true;
151             Serial.println(" OK");
152         } else {
153             Serial.printf(" fallo (rc=%d)\n", mqttClient.state());
154             delay(1000);
155         }
156     }
157 }
158
159 // Publicar telemetria a MQTT (-> Telegraf -> InfluxDB)
160 void publicarTelemetria() {
161     if (!mqttListo || !mqttClient.connected()) return;
162
163     // Construir el JSON que Telegraf leera
164     StaticJsonDocument<200> doc;
165     doc["rpm"] = round(rpm * 10.0f) / 10.0f;
166     doc["setpoint"] = setpoint;
167     doc["error"] = round((setpoint - rpm) * 10.0f) / 10.0f;
168     doc["pwm"] = round(pwmSalida);
169     doc["nodo"] = "ESP32-Motor";
170
171     char buf[200];
172     serializeJson(doc, buf);
173     mqttClient.publish(TOPIC_TELEMETRIA, buf);
174 }
175
176 // =====
177
178 void setup() {
179     Serial.begin(115200);
180
181     // Pines del motor
182     pinMode(AIN1, OUTPUT);
183     pinMode(AIN2, OUTPUT);
184     digitalWrite(AIN1, LOW);
185     digitalWrite(AIN2, LOW);
186
187     // Sensor Hall
188     pinMode(Amarillo, INPUT_PULLUP);
189     pinMode(Azul, INPUT_PULLUP);
190     attachInterrupt(digitalPinToInterrupt(Amarillo), countPulses, RISING);
191
192     // WiFi y MQTT (el PID arranca aunque falle la conexion)
193     conectarWiFi();
194     if (WiFi.status() == WL_CONNECTED) {
195         mqttClient.setServer(MQTT_BROKER, MQTT_PORT);
196         mqttClient.setCallback(mqttCallback);
197         mqttClient.setBufferSize(512);

```

```

198     conectarMQTT();
199 }
200
201 previousMillis = millis();
202 tMQTT = millis();
203
204 Serial.println("Sistema listo");
205 Serial.printf("Setpoint inicial: %.1f RPM\n", setpoint);
206 }
207
208 void loop() {
209     // Mantener conexion MQTT activa
210     if (mqttListo) mqttClient.loop();
211
212     unsigned long currentMillis = millis();
213
214     // 1. Loop de Control PID (cada 20 ms estrictos)
215     if (currentMillis - previousMillis >= interval) {
216         ejecutarCicloPID();
217         previousMillis = currentMillis;
218     }
219
220     // 2. Loop de Publicación a MQTT (cada 500 ms)
221     if ((currentMillis - tMQTT) >= T_MQTT_MS) {
222         publicarTelemetria();
223         tMQTT = currentMillis;
224
225         // Reconectar MQTT si se cayo
226         if (mqttListo && !mqttClient.connected()) {
227             conectarMQTT();
228         }
229     }
230 }

```

5.3. Parámetros del PID

Parámetro	Valor usado	Cómo lo obtuviste
K_p	1.5	Sacando la función de transferencia + MATLAB + Pruebas
K_i	12.5	Sacando la función de transferencia + MATLAB + Pruebas
K_d	0	-
Intervalo PID (ms)	20	Decisión nuestra
Intervalo MQTT (ms)	500	Parametro proyecto
PPR (pulsos/vuelta)	11	Data sheet

Table 3: Parámetros del sistema de control

6. Resultados

6.1. Capturas de pantalla del sistema funcionando

Instrucción: Inserta capturas de pantalla de cada uno de los siguientes elementos. Para cada una, escribe una oración describiendo qué muestra.

6.1.1. Serial Monitor del ESP32

```
Conectando a WiFi 'LaptopEdu'....  
OK IP: 192.168.137.82  
Conectando MQTT... OK  
Sistema listo  
Setpoint inicial: 70.0 RPM  
<-- Nuevo setpoint: 110.0 RPM  
<-- Nuevo setpoint: 40.0 RPM
```

Figure 5: Serial Monitor

Descripción: Serial Monitor: Se muestra el Monitor Serial del ESP32 durante el arranque del sistema, se puede observar el Wi-Fi al que se conecta, su dirección IP, si la conexión a MQTT se realizó de forma correcta y el setpoint inicial, posteriormente se muestran los setpoints enviados de manera remota por medio de MQTT.

6.1.2. Data Explorer de InfluxDB con los datos del motor

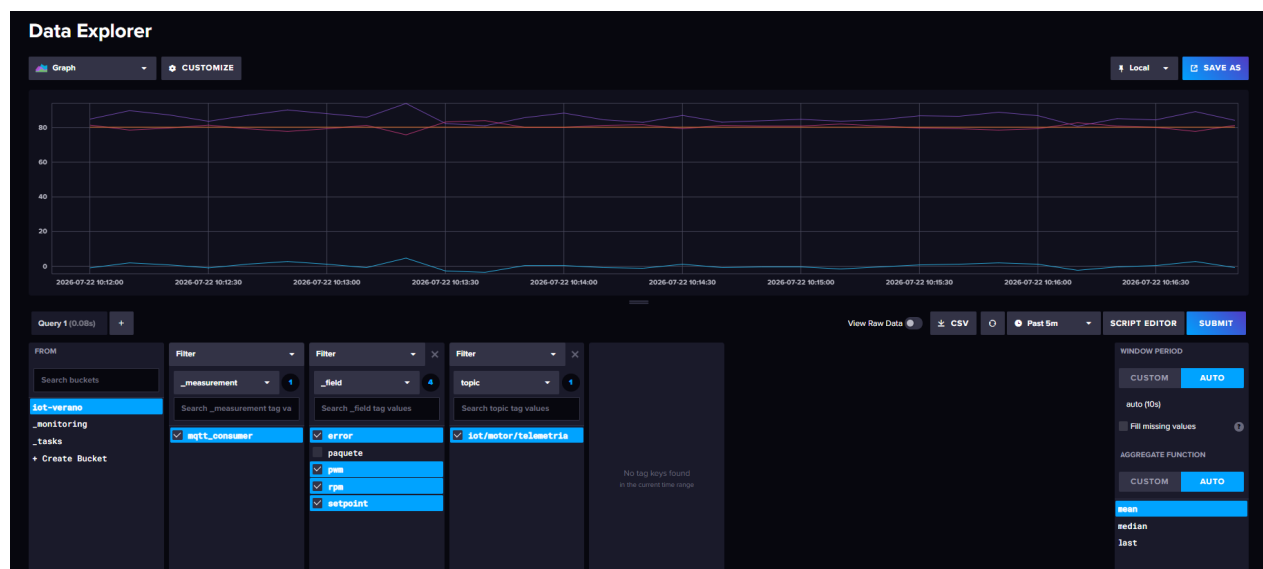


Figure 6: Data Explorer InfluxDB

Descripción: Se muestra el Data Explorer de InfluxDB, donde se visualizan las variables registradas del motor (RPM, setpoint, error y PWM). En esta gráfica podemos ver que todos los datos tienen su timestamp.

6.1.3. Dashboard completo de Grafana



Figure 7: Dashboard Grafana

Descripción: Se muestra el dashboard el cual hace uso de los gráficos de grafana que nos permiten visualizar toda nuestra información individualmente (RPM, setpoint, error y PWM).

6.2. Consultas Flux utilizadas

Instrucción: Pega las consultas Flux reales que usaste en cada panel de Grafana.

6.2.1. Panel RPM + Setpoint

Listing 3: Consulta Flux — Panel RPM y Setpoint

```
from(bucket: "iot-verano")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r["_field"] == "rpm" or r["_field"] == "setpoint")
  |> aggregateWindow(every: v.windowPeriod, fn: mean, createEmpty: false)
  |> yield(name: "motor")
```

6.2.2. Panel Error del PID

Listing 4: Consulta Flux — Panel Error

```
from(bucket: "iot-verano")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r["_field"] == "error")
  |> aggregateWindow(every: v.windowPeriod, fn: mean, createEmpty: false)
  |> yield(name: "error_pid")
```

6.2.3. Panel Señal PWM

Listing 5: Consulta Flux — Panel PWM

```
from(bucket: "iot-verano")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r["_field"] == "pwm")
  |> aggregateWindow(every: v.windowPeriod, fn: mean, createEmpty: false)
  |> yield(name: "pwm_control")
```

6.3. Experimento de escalón — datos de Grafana

Instrucción: Completa la tabla con los valores reales que observaste en Grafana durante el experimento de escalón. Esperar al menos 20 segundos en cada setpoint antes de anotar.

Setpoint (RPM)	RPM estado estacionario	Error residual	PWM estacionario	Observación
40	31 - 47	-7 - 9	46 - 70	Comportamiento esperado
80	70 - 85	-5 - 10	71 - 105	Comportamiento esperado
120	109 - 133	-13 - 11	133 - 164	Comportamiento esperado
170	140 - 164	6 - 30	255	Con la fuente de 12V no alcanza los RPM

Table 4: Respuesta del PID a distintos setpoints — datos de Grafana

6.4. Registro del Serial Monitor

Instrucción: Copia un fragmento representativo del Serial Monitor que incluya: arranque del sistema, conexión WiFi y MQTT, al menos 5 ciclos de publicación y la recepción de un nuevo setpoint.

Listing 6: Fragmento del Serial Monitor

```
1 Conectando a WiFi 'LaptopEdu'....
2 OK IP: 192.168.137.82
3 Conectando MQTT... OK
4 Sistema listo
5 Setpoint inicial: 70.0 RPM
6 <-- Nuevo setpoint: 80.0 RPM
7 <-- Nuevo setpoint: 0.0 RPM
8 <-- Nuevo setpoint: 130.0 RPM
9 <-- Nuevo setpoint: 80.0 RPM
10 <-- Nuevo setpoint: 110.0 RPM
```

7. Análisis y discusión

Instrucción: No repitas los resultados — interprétalos. Explica el **por qué** de lo que observaste. Mínimo 2 oraciones por pregunta.

7.1. Análisis del error residual en estado estacionario

Observando la tabla del experimento de escalón: ¿el motor llega exactamente al setpoint (error = 0)? ¿Cuál de los términos del PID es el responsable de eliminar el error residual y por qué?

Llega en momentos, pero nunca se estabiliza (por el funcionamiento del encoder), y el que corrige el error es la ki.

7.2. Análisis del sobreimpulso

¿Observaron sobreimpulso al cambiar el setpoint de 40 a 120 RPM? ¿Cómo se ve en la gráfica de Grafana? ¿Qué parámetro del PID reducirías para disminuirlo y qué consecuencia tendría en la velocidad de respuesta?

Hay un sobreimpulso de 20 RPM, y se podría reducir la ki para que no pase esto pero se estabilizaría mas lento.

7.3. Análisis de la señal PWM

¿El PWM es constante en estado estacionario o sigue variando? Explica por qué con base en los términos del PID. ¿Cuál de los 3 términos es el dominante en estado estacionario?

Sigue variando, ya que lo intenta mantener (como baja poco y sube poco los RPMs, este tiene que compensar, y el dominante seria la ki).

7.4. Latencia del pipeline

Cuando enviaron un nuevo setpoint por MQTT desde la terminal, ¿cuánto tiempo tardó en reflejarse el cambio en Grafana? Descompon el tiempo estimado en cada etapa del pipeline: terminal → MQTT → ESP32 → PID → motor → MQTT → InfluxDB → Grafana.

Etapa	Latencia estimada	Justificación
Terminal → Broker MQTT	$< 5ms$	Comunicación local por red/Docker en la misma PC
Broker → ESP32	$5 - 20ms$	Depende del momento exacto del ciclo en que llegue el mensaje Wi-Fi
ESP32 procesa el setpoint	$\leq 20ms$	Tiempo de muestreo configurado en el loop (interval = 20)
Motor responde (hasta 63%)	$\approx \tau$ del motor	Constante de tiempo mecánica característica del motor DC con caja reductora
ESP32 → InfluxDB (vía MQTT + Telegraf)	$10 - 50ms$	Tiempo de serialización JSON, transmisión y captura por Telegraf
InfluxDB → Grafana (refresco)	$500ms$	Intervalo configurado de publicación ($T_{MQTT} = 500$) y $refresh_interval = 500$)
Total percibido en Grafana	$\approx 550ms + \tau$	Suma del ciclo de telemetría más la inercia mecánica del motor

Table 5: Desglose de latencia del pipeline completo

7.5. ¿El PID está bien sintonizado? Tu criterio

Propón un criterio numérico propio para evaluar si tu PID está bien sintonizado, basado en los datos de Grafana. ¿Tu PID cumple ese criterio? Justifica con valores concretos de la tabla de resultados.

Mi criterio de sintonización: Que el error residual sea de ± 8 RPMs cuando ya esté estable (en 0.5 s) y haya pasado el sobreimpulso, ya que es lo que se ve cuando mandas un PWM constante.

Evaluación del PID contra el criterio: Si lo llega a cumplir (como se ve en al foto del Dashboard).

8. Preguntas de análisis técnico

Instrucción: Responde con fundamento técnico. Conecta cada respuesta con lo que observaste en la práctica.

- P1. En el código del PID, si eliminas el `integral = 0` del callback de setpoint y cambias de 40 a 170 RPM, ¿qué esperarías ver en la gráfica de Grafana durante los primeros segundos? Explica el fenómeno. El sistema no sería capaz de llegar al setpoint, se vería un sobreimpulso mayor y tardaría mucho en estabilizarse, si es que lo logra.
- P2. ¿Por qué el loop del PID corre cada 10 ms pero MQTT publica cada 500 ms? Si publicaras cada 10 ms (igual que el PID), ¿qué problemas podrían aparecer en el pipeline? Porque el PID requiere de datos en tiempo real y muy precisos para funcionar de la mejor manera, a diferencia de MQTT que no requiere de tanta información, pues solamente lo usamos para monitorear, mandarlos así de rápido saturaría de información “innecesaria”, haciendo todo más lento.
- P3. Escribe la consulta Flux que retorna el valor máximo de error que tuvo el motor en la última hora. Explica cómo la usarías para evaluar la calidad de tu sintonización.

```
from(bucket: "iot-verano")
  |> range(start: -1h)
  |> filter(fn: (r) => r["_field"] == "error")
  |> max()
```

Este valor de error máximo me ayudará a identificar la situación en la que el control falla más, con lo cual podría hacer ajustes a mi control para buscar una mejora en esas situaciones.

- P4. ¿Qué pasaría con el control del motor si el WiFi se cae mientras el motor está girando a 170 RPM? ¿Cómo está protegido el sistema ante esa situación en el código que escribieron? El control corre en la propia ESP por lo que debería seguir corriendo con el último setpoint indicado, simplemente no se mandaría la información.
- P5. El campo `nodo` en el JSON se convierte en Tag en InfluxDB. Si el sistema tuviera 10 motores con 10 ESP32 publicando al mismo topic, ¿cómo modificarías la consulta Flux para ver solo el motor de tu equipo?

```
from(bucket: "iot-verano")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r["_measurement"] == "mqtt_consumer" and r["nodo"] == "ESP32-Motor")
  |> filter(fn: (r) => r["_field"] == "rpm" or r["_field"] == "setpoint")
  |> aggregateWindow(every: v.windowPeriod, fn: mean, createEmpty: false)
  |> yield(name: "motor_equipo")
```

Agregaría el filtro anterior.

- P6. ¿Qué diferencia hay entre la frecuencia de control del PID (100 Hz en el ESP32) y la frecuencia de muestreo en Grafana? ¿Cuál es el límite teórico de la frecuencia de muestreo en Grafana con la configuración actual (refresco cada 5 s, intervalo MQTT 500 ms)?** La frecuencia de muestreo del control es muy veloz y la de grafana es únicamente la necesaria para visualizar de manera correcta la información, teóricamente podría tener la misma velocidad que el control pero esto resultaría en la saturación de la red y la base de datos

9. Conclusiones

Instrucción: Escribe mínimo **3 conclusiones numeradas**. Cada una debe: ser verificable con datos reales de la práctica, conectar con alguno de los conceptos del marco teórico, y no ser una opinión general.

1. Por medio de pruebas pudimos comprobar que el control PID funciona de manera correcta y veloz, respondiendo adecuadamente a variaciones en el sistema.
2. Pudimos comprobar que las mediciones de RPM, setpoint, error y PWM fueron transmitidas de manera correcta desde el ESP32 mediante MQTT, almacenadas en InfluxDB y visualizadas en Grafana en tiempo real
3. Por último se comprobó que el sistema es capaz de recibir instrucciones por medio de MQTT y realizarlas de manera correcta.

10. Repositorio y entregables

Lista de verificación antes de entregar:

- | | |
|--|---|
| ☒ PDF compilado sin errores | datos reales |
| ☒ Tabla de conexiones completa | ☒ Serial Monitor pegado |
| ☒ Código real pegado (sección 4) | ☒ 6 preguntas de análisis respondidas |
| ☒ Tabla de parámetros PID completa | ☒ Tabla de latencia del pipeline completa |
| ☒ 3 capturas de pantalla insertadas | ☒ Mínimo 3 conclusiones |
| ☒ Consultas Flux pegadas | ☒ Código y <code>.tex</code> en GitHub |
| ☒ Tabla del experimento de escalón con | |

Referencias

Instrucción: Mínimo 3 fuentes. Incluir el datasheet del TB6612FNG, la documentación de InfluxDB y al menos una fuente adicional.

References

- [1] InfluxData. (2026). Influxdata.Com. <https://docs.influxdata.com/>
- [2] Toshiba Bi-CD Integrated Circuit Silicon Monolithic TB6612FNG Driver IC for Dual DC motor. (n.d.).
- [3] JGA25-371 DC Gearmotor with Encoder (126 RPM at 12 V). (n.d.). https://people.ece.ubc.ca/~eng-services/files/courses/elec391-data_sheets/MOT4-info.pdf